**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ,
СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ
РОССИЙСКОЙ ФЕДЕРАЦИИ**

Федеральное государственное бюджетное образовательное учреждение высшего образования

Сибирский государственный университет телекоммуникаций и информатики

Кафедра телекоммуникационных систем и вычислительных средств (ТС и ВС)

РАСЧЕТНО-ГРАФИЧЕСКАЯ РАБОТА

Визуальное программирование и человеко -- машинное взаимодействие

Выполнил: студент гр. ИКС-431
Антропов Д. С.
Проверил:
Старший преподаватель
Ахпашев Р. В.

Новосибирск, 2026

Содержание

1 Введение **3**

2 Общая архитектура проекта **5**

2.1 Схема программно-аппаратного комплекса 6

3 Теоретическая часть **7**

3.1 Координаты LLA 7

3.2 Unix Epoch Time 8

3.3 GSM, LTE и NR 8

3.4 Критерии качества радиосигнала 9

4 Android-приложение **10**

4.1 Назначение и структура 10

4.2 Получение местоположения 10

4.3 Сбор параметров мобильной сети 11

4.4 Сбор сетевого трафика 11

4.5 Фоновый сервис 12

4.6 Формат JSON 12

5 ZeroMQ и клиент-серверное взаимодействие **13**

6 Backend-приложение на C++ **14**

6.1 Структура проекта 14

6.2 Многопоточность 14

6.3 Графический интерфейс 15

7 PostgreSQL **15**

8 OpenStreetMap **16**

9 Генерация тепловой карты **17**

10 Drive-test **18**

11 Заключение **19**

1. Введение

В рамках расчетно-графической работы был разработан программно-аппаратный комплекс для сбора, передачи, хранения и визуализации параметров мобильной сети. Комплекс состоит из Android-приложения и настольного backend-приложения на языке C++. Смартфон выступает в роли измерительного устройства: он получает координаты, параметры базовых станций, значения уровня радиосигнала и статистику сетевого трафика. Desktop-приложение принимает данные по сети, сохраняет их в файл и базу данных, строит графики, отображает маршрут на OpenStreetMap и формирует тепловые карты.

Практическая часть проекта объединяет задания по работе с местоположением смартфона, сокетами, ZeroMQ, сервисами Android, backend-сервером на C++, PostgreSQL, графическим интерфейсом ImGui/ImPlot, картографической подложкой OSM и алгоритмом построения heatmap методом обратных взвешенных расстояний. Основная цель проекта --- получить сквозной цикл обработки телеметрии: от получения измерения на смартфоне до анализа результата на компьютере.

Разработанное Android-приложение имеет основное меню, из которого пользователь может перейти к модулям калькулятора, медиаплеера и монитора. Для данной работы ключевым является модуль <<Монитор>>, который запускает foreground-service, получает координаты и данные сотовой сети, записывает накопленные измерения в JSON-файл либо отправляет их на сервер. Внешний вид главного меню приложения показан на рисунке 1.

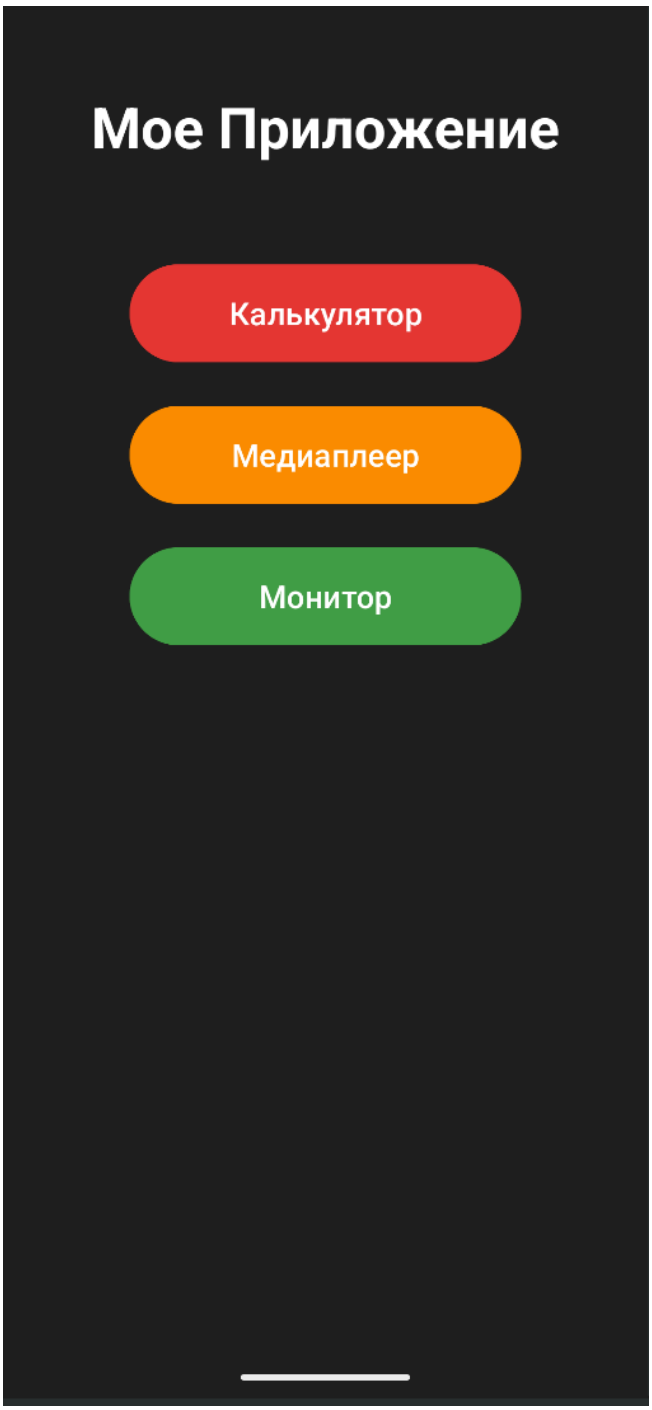


Рис. 1 – Главное меню Android-приложения

Backend-приложение называется Phone Monitor. Оно принимает JSON-пакеты от Android-клиента через ZeroMQ, парсит их, обновляет общую структуру данных, сохраняет информацию в PostgreSQL и отображает измерения в графическом интерфейсе. На рисунке 2 показана вкладка построения тепловой карты RSRP на фоне OpenStreetMap.

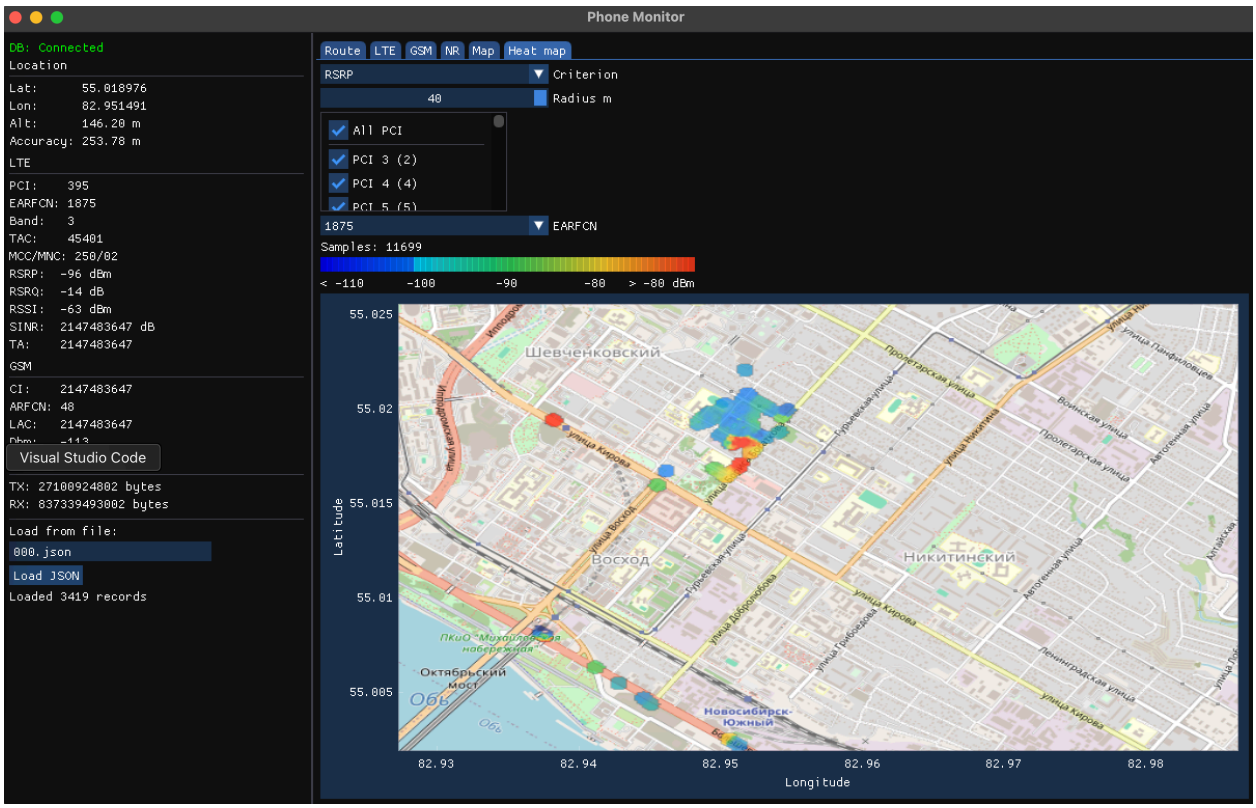


Рис. 2 – Desktop-интерфейс Phone Monitor с тепловой картой RSRP

2. Общая архитектура проекта

Комплекс построен по клиент-серверной модели. Android-приложение собирает телеметрию и формирует JSON-сообщение. Сообщение передается на компьютер по TCP с использованием паттерна ZeroMQ REQ/REP. Backend-сервер принимает пакет, отправляет ответ клиенту, сохраняет исходный JSON и обновляет состояние графического интерфейса.

В проекте используются следующие основные компоненты:

- Android-приложение на Kotlin;
- foreground-service для фонового сбора координат и параметров сети;
- ZeroMQ как транспортный слой между смартфоном и компьютером;
- C++20 backend-сервер;
- PostgreSQL для долговременного хранения измерений;
- ImGui и ImPlot для графического интерфейса и графиков;

- OpenStreetMap и libcurl для загрузки тайлов карты;
- алгоритм IDW для генерации тепловой карты.

2.1. Схема программно-аппаратного комплекса

На рисунке 3 представлена схема разработанного проекта. В верхней части находится Android-приложение, которое собирает GPS, CellInfo и traffic-данные. В центральной части расположено C++ backend-приложение, где выделены серверный поток, GUI-поток, модуль карты, модуль тепловой карты и база данных. В нижней части показано хранение данных в PostgreSQL и файловом кэше.

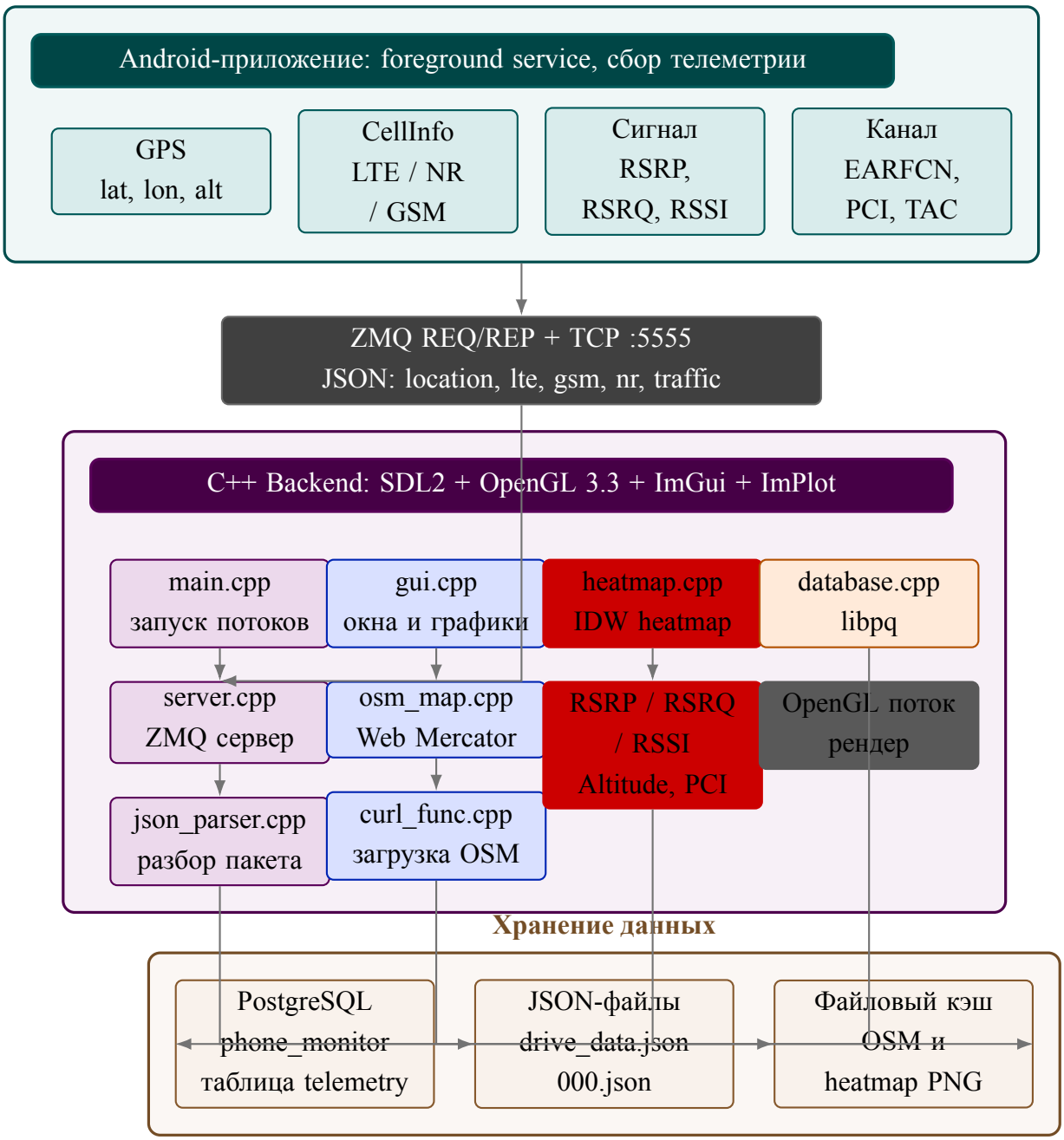


Рис. 3 – Архитектура программно-аппаратного комплекса

3. Теоретическая часть

3.1. Координаты LLA

В задаче мониторинга мобильной сети важны координаты точки измерения. В проекте используется представление LLA: latitude, longitude, altitude.

Latitude --- географическая широта. Она показывает положение точки относительно экватора и измеряется в градусах. Диапазон широты ограничен значениями от $-90°$ до $+90°$: отрицательные значения соответствуют южно-

му полушарию, положительные --- северному.

Longitude --- географическая долгота. Она показывает положение точки относительно нулевого меридиана и также измеряется в градусах. Диапазон долготы составляет от $-180°$ до $+180°$: отрицательные значения соответствуют западной долготе, положительные --- восточной.

Altitude --- высота над уровнем моря. В Android-классе `Location` высота возвращается в метрах. Значение зависит от источника определения местоположения: GPS, Wi-Fi, сотовые сети или их комбинация. Кроме координат Android также предоставляет accuracy --- оценку точности в метрах.

В программе координаты используются не только для вывода пользователю, но и для построения маршрута, привязки измерений к карте и интерполяции тепловой карты. Для каждого JSON-пакета сохраняются широта, долгота, высота, точность и время измерения.

3.2. Unix Epoch Time

Unix Epoch Time --- это способ представления времени как количества миллисекунд или секунд, прошедших с 00:00:00 UTC 1 января 1970 года. В Android для получения текущего времени используется метод `System.currentTi` который возвращает число миллисекунд с начала эпохи Unix.

В мобильных телефонах системное время может синхронизироваться по сети оператора, через NTP и по данным GNSS. В спутниковых системах навигации используется собственная шкала времени, отличающаяся от гражданского UTC, поэтому приемник выполняет пересчет. Точность времени в смартфоне зависит от источника синхронизации, качества приема спутникового сигнала, задержек сети и алгоритмов операционной системы.

3.3. GSM, LTE и NR

GSM --- цифровой стандарт мобильной связи второго поколения. В проекте для GSM сохраняются идентификатор соты, ARFCN, LAC, MCC, MNC и уровень сигнала в dBm. Эти данные позволяют определить, с какой сотой взаимодействовал телефон и насколько устойчивым был прием.

LTE --- стандарт четвертого поколения, использующий OFDMA в нисходящем канале и SC-FDMA в восходящем. Для LTE важны параметры `PCI`,

EARFCN, TAC, Band, RSRP, RSRQ, RSSI, RSSNR и Timing Advance. В drive-test анализе особенно важны RSRP и RSRQ, потому что они характеризуют покрытие и качество радиоканала.

NR --- стандарт пятого поколения. В Android для NR доступны NCI, PCI, NRARFCN, TAC, SS-RSRP, SS-RSRQ, SS-SINR. В проекте эти значения также включены в JSON и могут быть сохранены на стороне backend.

3.4. Критерии качества радиосигнала

RSSI показывает общий уровень принятой мощности в полосе канала. Этот параметр включает полезный сигнал, шум и интерференцию, поэтому сам по себе не всегда точно отражает качество LTE-соединения.

RSRP --- Reference Signal Received Power. Это мощность опорных сигналов LTE, измеряемая в dBm. RSRP используется для оценки покрытия. Чем ближе значение к нулю, тем сильнее сигнал. Например, -80 dBm лучше, чем -105 dBm.

RSRQ --- Reference Signal Received Quality. Этот параметр учитывает отношение RSRP к RSSI и позволяет оценивать качество приема с учетом загруженности и помех.

В проекте для тепловой карты RSRP используются следующие интервалы:

Таблица 1 – Критерии качества RSRP

Категория	Диапазон	Интерпретация
Excellent	> -80 dBm	сильный сигнал, максимальные скорости передачи данных
Good	$-80 \ldots -90$ dBm	надежная работа соединения
Fair/Marginal	$-90 \ldots -100$ dBm	соединение работоспособно, но возможны просадки скорости
Poor/Weak	$-100 \ldots -110$ dBm	возможны обрывы и низкая скорость
No Signal/Unusable	< -110 dBm	соединение часто недоступно

4. Android-приложение

4.1. Назначение и структура

Android-приложение находится в директории `/Users/m1/Desktop/work/` Основной пакет приложения --- `com.example.myapplication`. В проекте реализованы несколько Activity: главное меню, калькулятор, медиаплеер, модуль местоположения, модуль тестовой передачи ZMQ и модуль CellInfo.

Ключевые файлы Android-части:

- `MainActivity.kt` --- главное меню приложения;
- `LocationActivity.kt` --- интерфейс запуска мониторинга;
- `MonitorService` --- foreground-service для фонового сбора данных;
- `CellInfoActivity.kt` --- сбор CellInfo и отправка JSON на сервер;
- `SocketsZmqActivity.kt` --- простая проверка ZeroMQ REQ/REP;
- `AndroidManifest.xml` --- разрешения и регистрация Activity/Service.

В манифесте запрошены разрешения на доступ к интернету, точному и приблизительному местоположению, фоновой геолокации, состоянию телефона, статистике приложений и foreground-service. Это соответствует требованиям практик: приложение может получать координаты, параметры сотовой сети и работать в фоне.

4.2. Получение местоположения

Для получения координат используется `LocationManager`. В сервисе запускаются обновления от `GPS_PROVIDER` и `NETWORK_PROVIDER`. При поступлении нового значения вызывается `onLocationChanged()`, где сохраняется последнее известное местоположение. Далее каждые пять секунд сервис формирует JSON-пакет.

Фрагмент логики формирования объекта `location`:

Listing 1 – Формирование блока location в Android-сервисе

```
val loc = lastLocation
```

```
val t = System.currentTimeMillis()
if (loc != null)
    sb.append("\"location\":{\"latitude\":${loc.latitude},\"
   longitude\":${loc.longitude},\"altitude\":${loc.altitude},\"
   accuracy\":${loc.accuracy},\"time\":$t}")
else
    sb.append("\"location\":{\"latitude\":0,\"longitude\":0,\"
   altitude\":0,\"accuracy\":0,\"time\":$t}")
```

Для пользователя предусмотрено два режима работы. В онлайн-режиме данные отправляются на сервер. В офлайн-режиме JSON-пакеты записываются в файл `drive_data.json`, что удобно для drive-test, когда сервер временно недоступен.

4.3. Сбор параметров мобильной сети

Для получения информации о сотах используется `TelephonyManager.allC`
В проекте обрабатываются LTE, GSM и NR-соты.

Для LTE сохраняются:

- `band`, `ci`, `earfcn`, `mcc`, `mnc`;
- `pci`, `tac`;
- `asu`, `cqi`, `rsrp`, `rsrq`, `rssi`;
- `rssnr`, `ta`.

Для GSM сохраняются `ci`, `bsic`, `arfcn`, `lac`, `mcc`, `mnc`, `dbm`, `rssi` и `ta`. Для NR сохраняются `band`, `nci`, `pci`, `nrarfcn`, `tac`, `mcc`, `mnc`, `ss_rsrp`, `ss_rsrq`, `ss_sinr`.

4.4. Сбор сетевого трафика

Дополнительно приложение собирает суммарный объем переданных и принятых данных через `TrafficStats`. Для определения приложений с высоким потреблением трафика используется `NetworkStatsManager`. Сначала собирается статистика по установленным приложениям, затем вычисляются среднее значение и стандартное отклонение. В список попадают приложения, находящиеся выше порога $\mu + 2\sigma$.

Такой подход позволяет не выводить весь список приложений, а оставить только наиболее заметных потребителей трафика. Это полезно при анализе drive-test, потому что передача данных по сети зависит не только от качества радиосигнала, но и от активности приложений.

4.5. Фоновый сервис

Класс `MonitorService` реализован как foreground-service. Это позволяет продолжать запись измерений, когда Activity не находится на экране пользователя. Сервис запускает уведомление, получает обновления координат, формирует JSON и либо пишет его в файл, либо отправляет на backend.

Сервис использует корутины:

Listing 2 – Цикл периодической отправки данных

```
serviceScope.launch {
    while (isActive) {
        delay(5000)
        val json = buildJson()
        if (offline) {
            saveToFile(json)
            broadcast("Записано в файл", json)
        } else {
            broadcast("Отправка...", json)
            sendToServer(json)
        }
    }
}
```

Для связи Activity и Service используется broadcast. Activity получает статус и последний JSON-пакет, форматирует его и выводит на экран.

4.6. Формат JSON

Один пакет данных содержит координаты, список LTE/GSM/NR-сот и статистику трафика:

Listing 3 – Пример JSON-пакета Android-приложения

```
{
  "location": {
```

```
    "latitude": 55.018976,
    "longitude": 82.951491,
    "altitude": 146.2,
    "accuracy": 253.78,
    "time": 1773768390621
  },
  "lte": [
    {
      "band": 3,
      "ci": 138245901,
      "earfcn": 1875,
      "mcc": "250",
      "mnc": "02",
      "pci": 395,
      "tac": 45401,
      "rsrp": -96,
      "rsrq": -14,
      "rssi": -63,
      "rssnr": 2147483647
    }
  ],
  "traffic": {
    "total_tx": 27100924802,
    "total_rx": 837339493002,
    "top_apps": []
  }
}
```


5. ZeroMQ и клиент-серверное взаимодействие

ZeroMQ используется как библиотека обмена сообщениями поверх TCP. В проекте выбран шаблон REQ/REP: клиент отправляет запрос, сервер принимает JSON и отвечает строкой подтверждения. Такой паттерн прост для отладки и подходит для периодической отправки телеметрии.

На Android создается сокет `SocketType.REQ`, который подключается к адресу `tcp://<ip>:5555`. После отправки JSON клиент ожидает ответ. Если ответ не приходит за заданный таймаут, приложение сообщает об ошибке и повторяет попытку отправки.

На стороне C++ backend работает серверный поток. Он принимает сообщения, парсит JSON, обновляет глобальную структуру телеметрии и возвращает ответ клиенту. Отдельный поток нужен для того, чтобы прием данных не блокировал интерфейс ImGui.

6. Backend-приложение на C++

6.1. Структура проекта

Backend находится в директории `/Users/m1/Desktop/visual`. Проект собирается через CMake и использует C++20. Структура исходного кода разделена по назначению:

Таблица 2 – Основные файлы backend-приложения

Файл	Назначение
`main.cpp`	запуск приложения, создание серверного потока и GUI
`server.cpp/.h`	ZeroMQ-сервер, прием JSON-пакетов
`gui.cpp/.h`	графический интерфейс ImGui/ImPlot
`json_parser.cpp/.h`	разбор JSON и заполнение структур данных
`database.cpp/.h`	подключение к PostgreSQL и запись строк в таблицу
`osm_map.cpp/.h`	пересчет координат в тайлы OSM, работа с картой
`curl_func.cpp/.h`	загрузка PNG-тайлов через libcurl
`heatmap.cpp/.h`	генерация тепловой карты методом IDW
`types.h`	общие структуры телеметрии

6.2. Многопоточность

Графический интерфейс работает в основном потоке, потому что на macOS SDL2/OpenGL должны выполняться в main thread. Сервер ZeroMQ запускается в отдельном `std::thread`. Такое разделение позволяет интерфейсу оставаться отзывчивым даже во время приема данных.

В проекте также используются дополнительные фоновые вычисления. Загрузка OSM-тайлов выполняется асинхронно: если нужный тайл отсутствует в кэше, он скачивается через libcurl и затем декодируется в RGBA. Построение тепловой карты запускается отдельно через `std::async`, потому что IDW-интерполяция может занимать заметное время при большом количестве точек.

6.3. Графический интерфейс

Интерфейс Phone Monitor состоит из левой панели текущих значений и основной области с вкладками. Левая панель показывает состояние базы данных, координаты, параметры LTE/GSM/NR и статистику трафика. В основной области доступны вкладки:

- `Route` --- маршрут движения по координатам;
- `LTE` --- графики RSRP, RSRQ, RSSI, SINR по PCI;
- `GSM` --- графики уровня GSM;
- `NR` --- параметры 5G NR;
- `Map` --- маршрут на фоне OpenStreetMap;
- `Heat map` --- тепловая карта выбранного критерия.

Для графиков используется ImPlot. При наличии нескольких LTE-сот значения отображаются разными линиями, а в легенду добавляются PCI. Это позволяет сравнивать качество приема от разных сот в одной временной области.

7. PostgreSQL

Backend подключается к базе данных `phone_monitor`. При запуске создается таблица `telemetry`, если она еще не существует. В таблицу сохраняются координаты, traffic-статистика, основные параметры GSM/LTE/NR и исходный JSON-пакет.

Таблица 3 – Схема таблицы telemetry

Поле	Тип	Назначение
id	SERIAL	первичный ключ
received	TIMESTAMP	время приема на сервере
lat, lon	DOUBLE PRECISION	координаты точки измерения
alt, accuracy	REAL	высота и точность
gps_time	BIGINT	время Android в формате Unix Epoch
total_tx, total_rx	BIGINT	суммарный трафик
gsm_ci, gsm_dbm	INTEGER	параметры GSM
lte_pci, lte_rsrp	INTEGER	PCI и RSRP LTE
lte_rsrq, lte_rssi	INTEGER	качество и общий уровень LTE
lte_sinr, lte_band	INTEGER	SINR и band LTE
nr_pci, nr_rsrp, nr_sinr	INTEGER	параметры NR
raw_data	JSONB	исходный JSON-пакет

Использование поля raw_data удобно для расширения проекта. Даже если в таблицу пока вынесены не все поля, полный пакет остается доступен для последующего анализа.

8. OpenStreetMap

Для отображения карты используется тайловая модель OpenStreetMap. Поверхность Земли представляется в проекции Web Mercator. При заданном масштабе zoom вся карта делится на квадратные тайлы размером 256 на 256 пикселей. Номер тайла вычисляется по широте и долготе:

$$x = \left\lfloor \frac{\lambda + 180}{360} \cdot 2^z \right\rfloor,$$

$$y = \left\lfloor \frac{1 - \ln(\tan\varphi + \sec\varphi)/\pi}{2} \cdot 2^z \right\rfloor,$$

где λ --- долгота в градусах, φ --- широта в радианах, z --- уровень масштабирования.

Backend определяет набор тайлов, которые нужны для текущих границ окна ImPlot, и загружает их через libcurl. Загруженные файлы сохраняются в кэше:

Listing 4 – Структура кэша OSM-тайлов

```
build/
  zoom/
    x/
      y.png
```

Если тайл уже был загружен, он читается с диска. Если файла нет, приложение скачивает его с `tile.openstreetmap.org`. PNG декодируется в RGBA-массив и передается в OpenGL-текстуру, после чего отображается в ImPlot через `PlotImage`.

9. Генерация тепловой карты

Тепловая карта строится методом обратных взвешенных расстояний, IDW. Метод основан на предположении, что значение в неизвестной точке сильнее зависит от ближайших измерений, чем от дальних. Для вычисляемой точки (x, y) значение определяется формулой:

$$V(x,y) = \frac{\sum\limits_{i=1}^{n} w_i V_i}{\sum\limits_{i=1}^{n} w_i}, \quad w_i = \frac{1}{d_i^p},$$

где V_i --- измеренное значение, d_i --- расстояние до точки измерения, p

--- степень влияния расстояния. В проекте учитываются только измерения в радиусе от 10 до 40 метров. Это соответствует заданию и уменьшает влияние удаленных точек.

Heatmap поддерживает критерии:

- RSRP;
- RSRQ;
- RSSI;
- Altitude.

Пользователь выбирает критерий, EARFCN, PCI и радиус интерполяции. Также можно включить режим `All PCI`, в котором карта строится по всем сотам выбранного EARFCN. Для RSRP нормализация выполняется в диапазоне от -110 до -80 dBm: значения хуже -110 dBm считаются непригодными и не закрашиваются, а значения лучше -80 dBm соответствуют отличному уровню сигнала.

Цветовая шкала в интерфейсе соответствует критериям задания: темно-синие оттенки обозначают слабый сигнал, зеленые и желтые --- средний уровень, красный --- сильный сигнал. Сгенерированная картинка сохраняется в директории запуска:

Listing 5 – Имя файла тепловой карты

```
build/heatmap_<criterion>_pci_<pci>_earfcn_<earfcn>.png
```

10. Drive-test

Drive-test выполняется в городской местности. Смартфон запускается в режиме мониторинга, приложение записывает координаты и параметры сети в JSON-файл. Далее файл загружается в backend через кнопку `Load JSON`. После загрузки восстанавливается маршрут, история LTE/GSM/NR и графики уровня сигнала.

В представленном наборе данных загружено 3419 записей. В окне backend отображаются координаты, текущие значения LTE и GSM, количество принятых и отправленных байтов, выбранный EARFCN 1875 и тепловая карта

RSRP. По карте видно, что значения сигнала меняются вдоль маршрута: отдельные участки окрашены в красный и желтый цвет, а зоны более слабого приема --- в синий и зеленый.

11. Заключение

В ходе работы был создан программно-аппаратный комплекс, который объединяет мобильное приложение и desktop-backend. Android-приложение получает координаты, параметры LTE/GSM/NR, статистику трафика и может работать в фоне через foreground-service. Backend-приложение принимает данные через ZeroMQ, сохраняет их в PostgreSQL и JSON, отображает текущие значения, строит графики, показывает маршрут на OpenStreetMap и формирует тепловые карты.

За учебный год были изучены основные этапы разработки сетевого измерительного комплекса: работа с разрешениями Android, получение GPS и CellInfo, передача данных по сокетам, использование ZeroMQ, создание C++ backend-сервера, многопоточность, подключение PostgreSQL, построение интерфейса на ImGui, работа с PNG-тайлами и интерполяция измерений методом IDW. Итоговый проект можно использовать как основу для анализа покрытия мобильной сети и проведения drive-test измерений.

Список литературы

[1] Android Developers. Location overview. URL: https://developer.android.com/develop/sensors-and-location/location

[2] Android Developers. TelephonyManager and CellInfo API. URL: https://developer.android.com/reference/android/telephony/TelephonyManager

[3] ZeroMQ Documentation. URL: https://zeromq.org/documentation/

[4] PostgreSQL Documentation. URL: https://www.postgresql.org/docs/

[5] Dear ImGui GitHub repository. URL: https://github.com/ocornut/imgui

[6] ImPlot GitHub repository. URL: https://github.com/epezent/implot

[7] OpenStreetMap Wiki. Slippy map tilenames. URL: https://wiki.openstreetmap.org/wiki/Slippy_map_tilenames

[8] 3GPP TS 36.214. E-UTRA Physical layer measurements. URL: https://www.3gpp.org/